which the second character is a punctuation character. These predefined variables are listed Predefined Variables, on page 451. Finally, a global variable name can be formed using $- followed by a single letter or underscore. These latter variables typically mirror the setting of the corresponding command-line option (see Execution Environment Variables, on page 451 for details):

```
$params  $PROGRAM  $!  $_  $-a  $-K
```

Variable/Method Ambiguity

When Ruby sees a name such as a in an expression, it needs to determine whether it is a local variable reference or a call to a method with no parameters. To decide which is the case, Ruby uses a heuristic. As Ruby parses a source file, it keeps track of symbols that have had values assigned to them. Ruby assumes that these symbols are variables. When it subsequently comes across a symbol that could be a variable or a method call, it checks to see whether it has seen a prior assignment to that symbol. If so, it treats the symbol as a variable; otherwise, it treats it as a method call.

As a somewhat pathological case of this, consider the following code fragment, submitted by Clemens Hintze:

```
def a
  puts "Function 'a' called"
  99
end

(1..2).each do |i|
  if i == 2
    puts "i==2, a=#{a}"
  else
    a = 1
    puts "i==1, a=#{a}"
  end
end
```

produces:

```
i==1, a=1
Function 'a' called
i==2, a=99
```

When parsing the file, Ruby sees the use of a in the puts statement in the main branch of the if and, because it hasn't yet seen any assignment to a, assumes that a is a method call. By the time it gets to the puts statement in the else branch, though, it *has* seen an assignment and so treats a as a variable.

Note that the assignment does not have to be executed—Ruby just has to have seen it. This program does not raise an error.

```
a = 1 if false; a # => nil
```

Variables and Constants

Ruby variables and constants hold references to objects. Variables themselves do not have an intrinsic type. Instead, the type of a variable is defined solely by the messages to which the object referenced by the variable responds. (When we say that a variable is not typed,